Computação Gráfica

Ciência da Computação | Engenharia da Computação

Prof.: Rafael Peiter | rpeiter@unisc.br

UNISC
UNIVERSIDADE DE SANTA CRUZ DO SUL

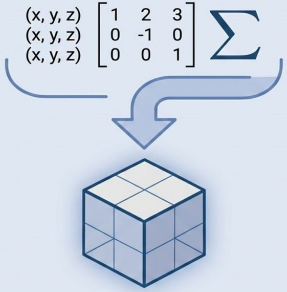
UNISC
UNIVERSIDADE DE SANTA CRUZ DO SUL

Computação Gráfica
Aula 02 - APIs gráficas

Departamento de Informática - UNISC
Professor: Rafael Peiter
rpeiter@unisc.br

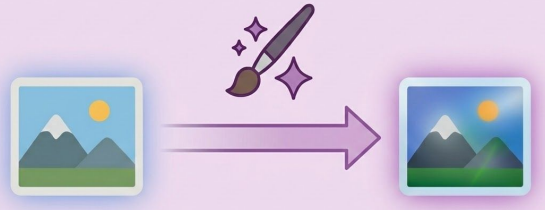
Aula passada

Pilar 1: Geração de Imagens (Síntese / CG)



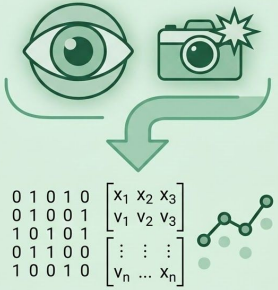
De dados matemáticos e modelos,
CRIAR imagens visuais e mundos.

Pilar 2: Processamento de Imagens



De imagens existentes,
MELHORAR ou ALTERAR imagens.

Pilar 3: Visão Computacional (Análise / PI)



De imagens visuais e capturas, EXTRAIR
dados matemáticos e COMPREENDER mundos.

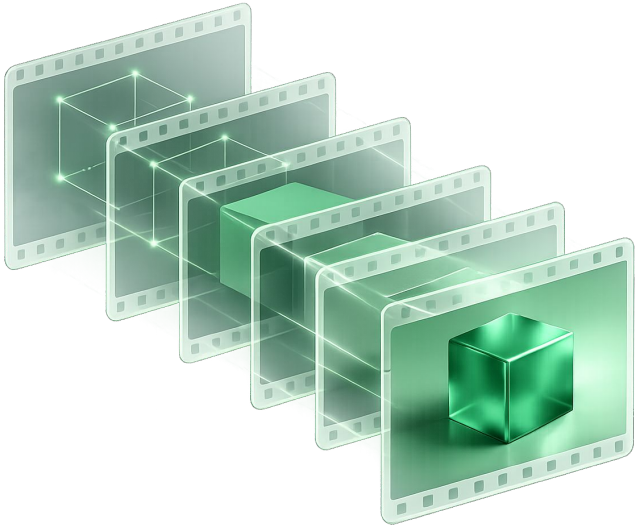
Tópicos

1. APIs gráficas
2. OpenGL
3. Exercícios

A ilusão da vida

Em gráficos de computador o movimento não existe

- Cenas animadas são apenas uma sequência de imagens estáticas exibidas em rápida sucessão
- Nosso cérebro preenche as lacunas, interpretando objetos mudando de aparência ou posição de forma contínua
- Cada imagem individual gerada é chamada de **Quadro (Frame)**.



APIs gráficas

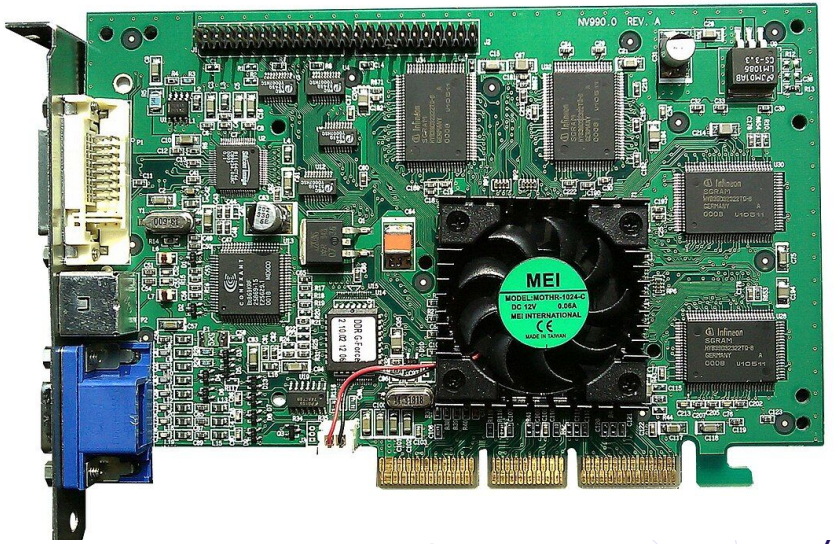
- No início dos anos 1990, os computadores dependiam da (CPU) para realizar os cálculos necessários para gráficos.

- À medida que gráficos 3D em tempo real tornaram-se cada vez mais comuns em plataformas de jogos de vídeo (incluindo arcades, consoles de jogos e computadores pessoais), houve uma demanda crescente por hardware especializado para renderizar esses gráficos.

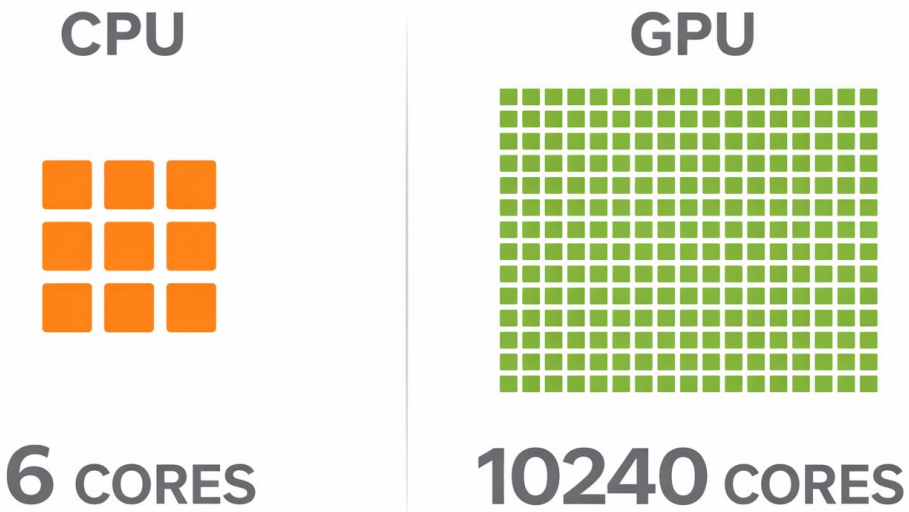
- Isso levou ao desenvolvimento da unidade de processamento gráfico (GPU)

APIs gráficas

- O termo **GPU** foi popularizado pela Nvidia Corporation em 1999 com o lançamento do **GeForce 256**, um processador de chip único que realiza transformações geométricas e cálculos de iluminação



APIs gráficas

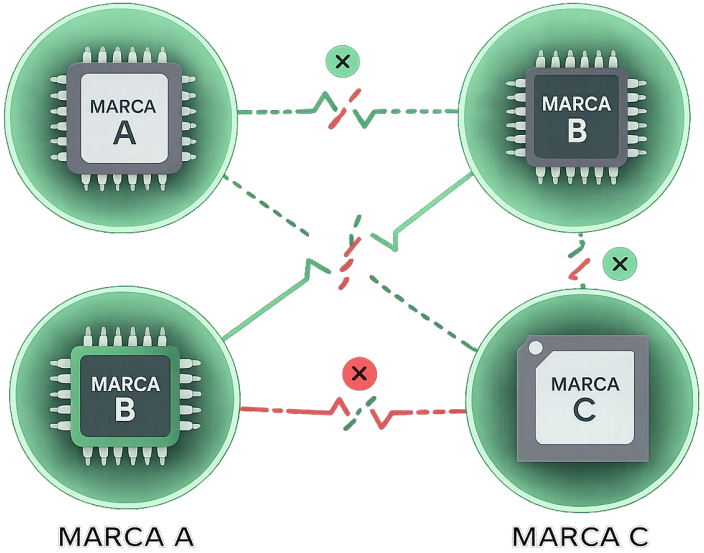


APIs gráficas

- Cada fabricante de hardware tinha suas próprias instruções para desenho 2D e 3D

- Com o tempo houve uma padronização e atualmente existem algumas APIs

- Para que serve?
  - **Abstração** da **complexidade** do hardware gráfico



DirectX

O **DirectX** é uma **API** gráfica projetada para Windows que especifica uma interface de software padrão para hardware de processamento de gráficos 3D

DirectX

- Projetado para Windows e Xbox
- Otimizado para hardware de jogos

Metal

- API de gráficos para iOS e macOS
- Exclusivo para hardware da Apple

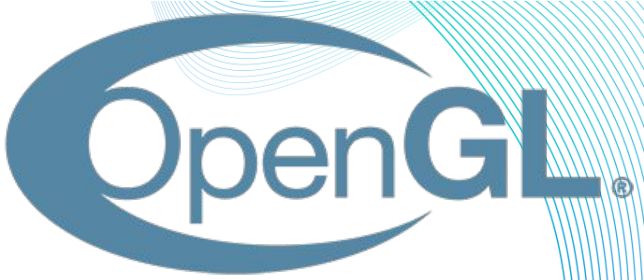
WebGL

Uma **API** de gráficos 3D que é implementada em **navegadores** sem a necessidade de plugins

Vulkan

- API de gráficos de baixo nível
- Maior eficiência e desempenho
- Mesmo grupo que gerencia o OpenGL
- Considerado o sucessor do OpenGL

OpenGL



O **OpenGL** é uma **API** gráfica de várias plataformas que especifica uma interface de software padrão para hardware de processamento de gráficos 3D

OpenGL

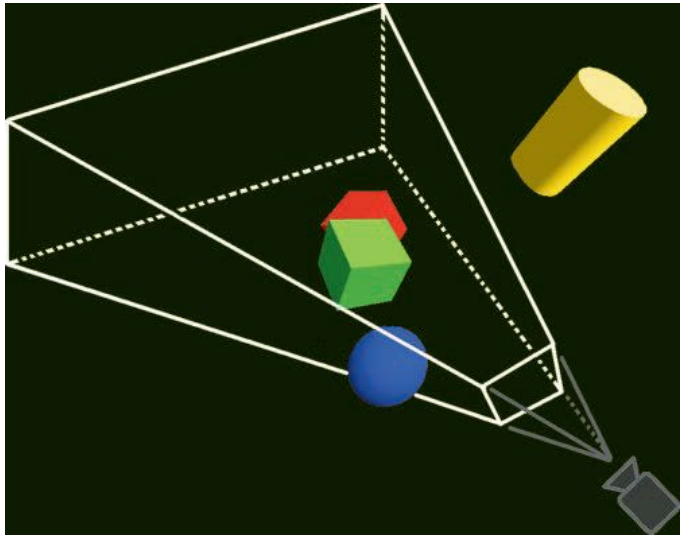
- Multiplataforma
- Padrão aberto gerenciado pelo grupo Khronos
- Utilizado em CAD, realidade virtual, jogos, etc
- O OpenGL é suficientemente versátil para suportar projetos desde o nível básico até o avançado
- As APIs e bibliotecas utilizadas na disciplina são para várias linguagens e multiplataforma

Conceitos

- O nosso objetivo principal é **gerar imagens bidimensionais** de cenas tridimensionais, esse processo é chamado de **renderização da cena**

- Esses objetos podem parecer ter apenas uma cor ou sua aparência pode ser afetada por texturas, fontes de luz que resultam em sombreamento e sombras, ou propriedades ambientais como neblina

Conceitos



- As cenas são renderizadas do ponto de vista de uma **câmera virtual**, cuja posição relativa e orientação na cena, juntamente com suas propriedades intrínsecas, como **ângulo de visão** e **profundidade** de campo, **determinam quais objetos serão visíveis** ou parcialmente obscurecidos por outros objetos quando a cena é renderizada

Conceitos

- De uma perspectiva técnica e mais detalhada, renderizar uma cena produz um **raster** — um **arranjo de pixels** (elementos de imagem) que serão exibidos em uma tela, organizados em uma grade bidimensional.

- Os pixels são tipicamente extremamente pequenos, ampliar uma imagem pode ilustrar a presença de pixels individuais

Conceitos

- Os pixels especificam cores utilizando ponto flutuante entre 0 e 1 para representar a quantidade de luz vermelha, verde e azul presente em uma cor

	R	G	B
red	1	0	0
orange	1	0.5	0
yellow	1	1	0
green	0	1	0
blue	0	0	1
violet	0.5	0	1

	R	G	B
black	0	0	0
white	1	1	1
gray	0.5	0.5	0.5
brown	0.5	0.2	0
pink	1	0.5	0.5
cyan	0	1	1

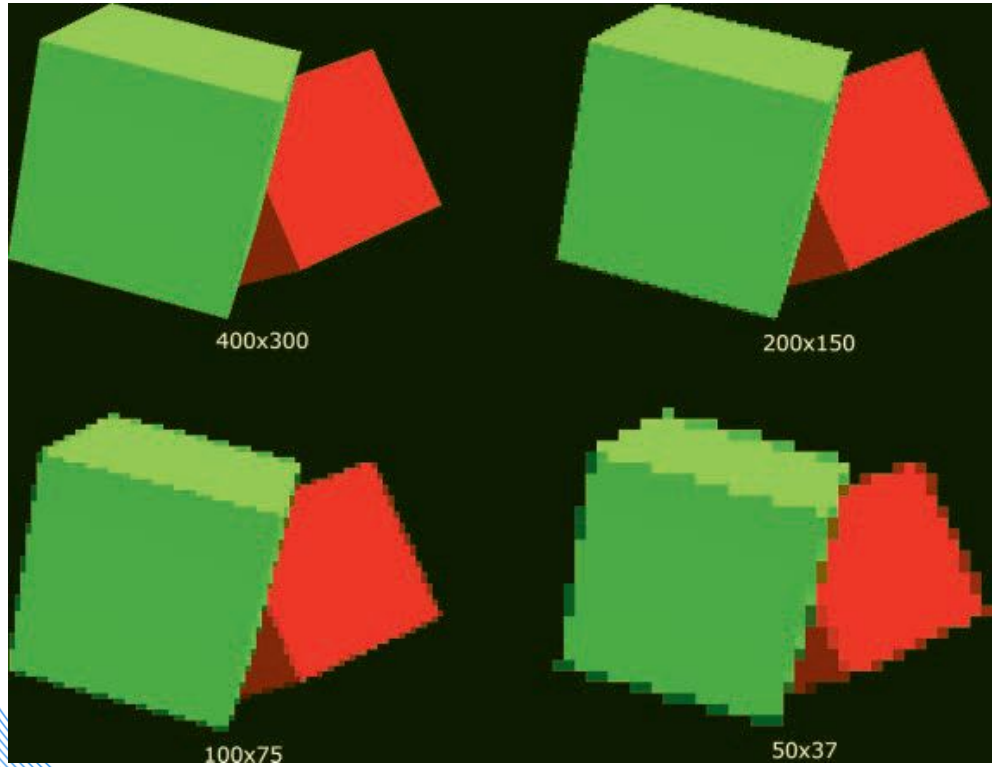
Conceitos

- A qualidade de uma imagem depende em parte de sua **resolução** (o número de pixels no raster) e **precisão** (o número de bits usados para cada pixel). Como cada bit tem dois valores possíveis (0 ou 1), o número de cores que podem ser expressas com precisão de N bits é 2^N

- Atualmente, o padrão de profundidade de cor mais utilizado é de 24 bits, também conhecido como **True Color**, que oferece 16,7 milhões de cores (8 bits por canal de cor, para os canais vermelho, verde e azul).

- Padrão para maioria dos sistemas operacionais, monitores e aplicações, devido ao seu bom equilíbrio entre a qualidade da imagem e o uso eficiente dos recursos do sistema.

Conceitos



Double Buffer

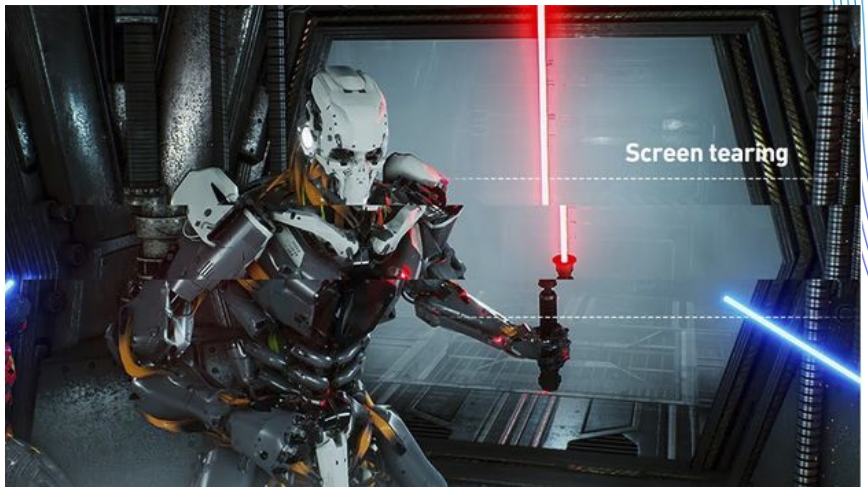
A cena é primeiramente renderizada off-screen em um buffer não visível ao usuário (chamado de **back buffer**). Quando a renderização está completa, o back buffer é trocado com o front buffer (o buffer que está sendo atualmente exibido na tela).

Double Buffering

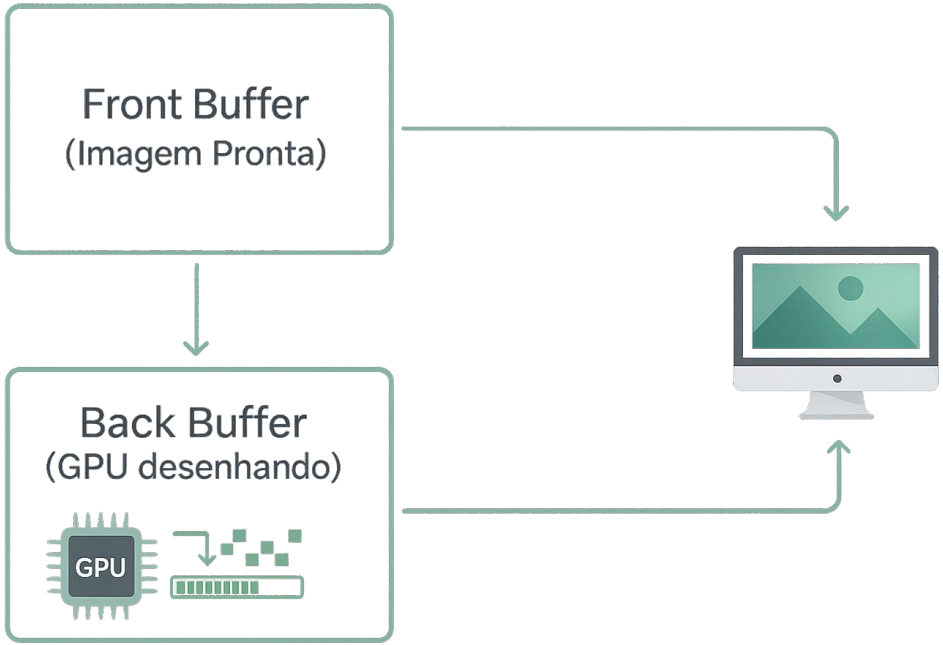
Double Buffering é uma técnica usada para minimizar ou eliminar o **tearing, flickering** e outros artefatos visuais durante a renderização de gráficos em movimento.

Double Buffering

Efeito **tearing**

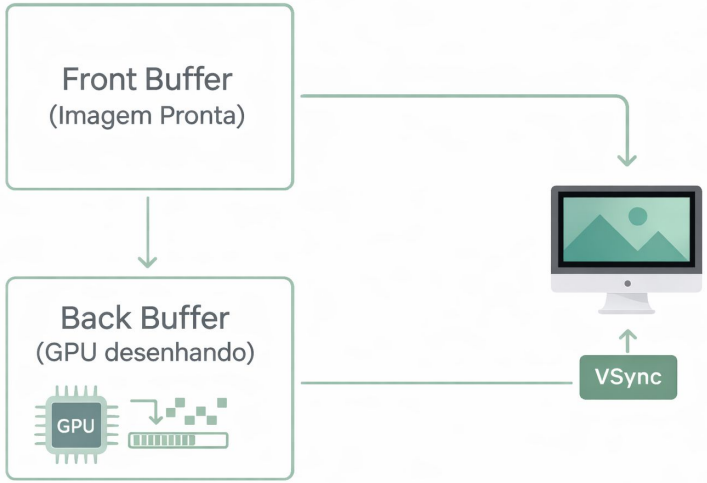


Double Buffering



Vsync

Assegura que a troca entre o **front buffer** e o **back buffer** ocorra no **momento apropriado**, alinhando a atualização do buffer de exibição com o ciclo de atualização do monitor



Conceitos

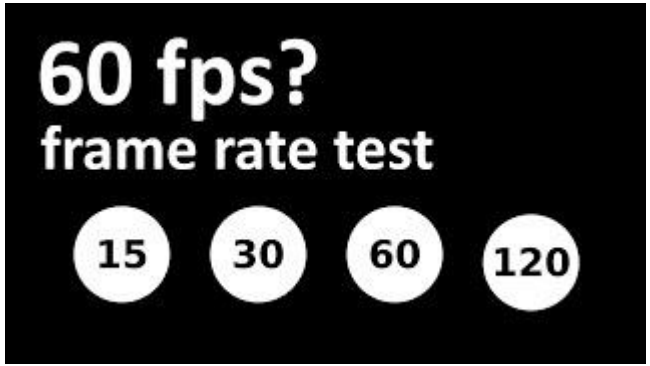
- Além de renderizar cenas tridimensionais, outro objetivo em gráficos de computador é **criar cenas animadas**.

- Animações consistem em uma sequência de imagens exibidas em sucessão rápida o suficiente para que o espectador interprete os objetos nas imagens como se estivessem se movendo ou mudando de aparência continuamente.

- Cada imagem que é exibida é chamada de **quadro ou frame**. A velocidade com que estas imagens aparecem é chamada de taxa de quadros e é medida em **quadros por segundo (FPS)**.

Conceitos

- A taxa de quadros padrão para filmes e televisão é de **24 FPS**. Monitores de computador tipicamente exibem gráficos a **60 FPS**. Para simulações de realidade virtual, os desenvolvedores visam alcançar **90 FPS**, pois taxas de quadros mais baixas podem causar desorientação e outros efeitos negativos nos usuários.



Pipeline gráfico

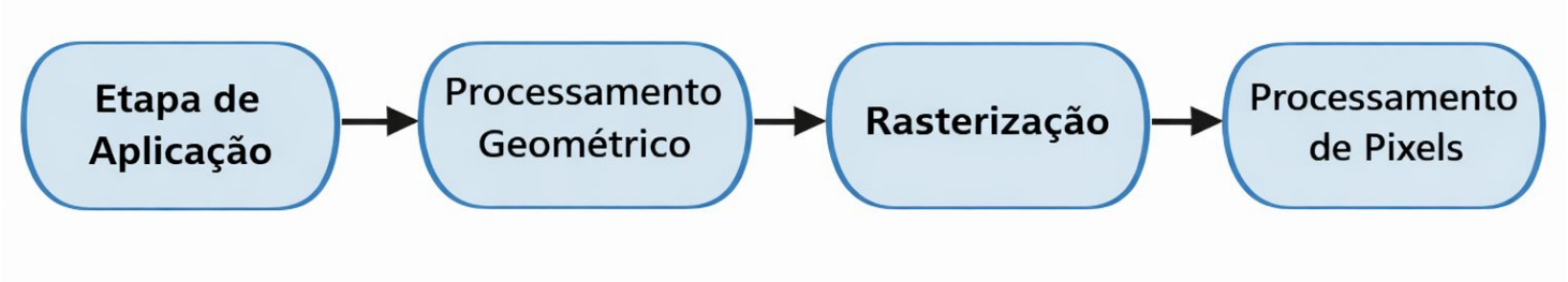
- Um pipeline gráfico é um modelo que descreve uma sequência de etapas necessárias para renderizar uma cena tridimensional

- A utilização de pipeline permite que uma tarefa computacional seja dividida em subtarefas, cada uma das quais pode ser trabalhada em paralelo, de maneira semelhante a uma linha de montagem para fabricar produtos em uma fábrica, o que aumenta a eficiência geral.

- Pipelines gráficos aumentam a eficiência do processo de renderização, permitindo que imagens sejam exibidas em taxas mais rápidas.

Pipeline gráfico

- Vários modelos de pipeline são possíveis, em OpenGL existem quatros estágios que são utilizados para renderizar gráficos em tempo real



Estágio de aplicação

- Inicializa a janela onde os gráficos renderizados serão exibidos, enviando dados para a GPU

- No caso de aplicações animadas ou interativas, a aplicação principal contém um loop que renderiza a cena repetidamente, visando tipicamente uma taxa de 60 FPS

- Outros processos que podem ser gerenciados pela CPU incluem o monitoramento de hardware para eventos de entrada do usuário, ou a execução de algoritmos para tarefas como simulação física e detecção de colisão.

Processamento geométrico

Determina a **posição** de cada **vértice** das formas geométricas a serem renderizadas, implementado por um programa chamado **vertex shader**

Processamento geométrico

- Em CG, a forma de um objeto geométrico é definida por uma malha: uma coleção de pontos que são agrupados em linhas ou triângulos

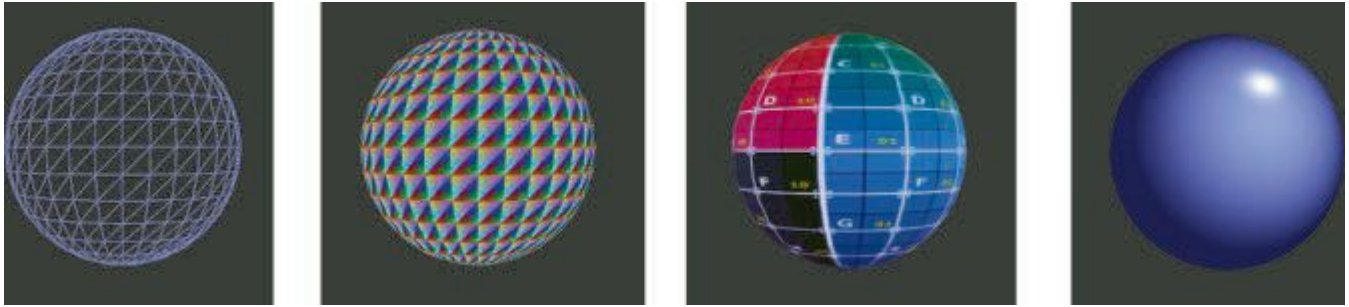
Processamento geométrico

- Além da forma geral de um objeto, informações adicionais podem ser necessárias para descrever como o objeto deve ser renderizado

- As propriedades ou atributos que são específicos para a renderização de cada ponto individual são agrupados em uma estrutura de dados chamada **vertex (vértice)**

Processamento geométrico

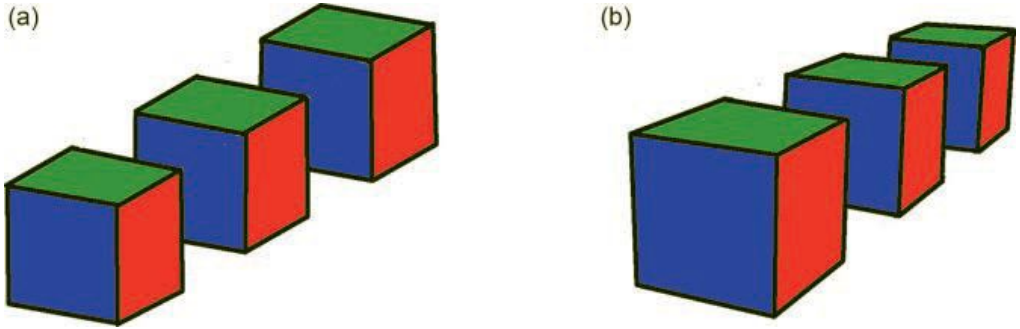
- Especificação das cores e texturas



Processamento geométrico

- Toda a etapa de projeção da cena também é feita aqui

- Uma projeção perspectiva é considerada para produzir imagens mais realistas, já que objetos que estão mais distantes da câmera virtual requerem maior compressão pela transformação e, assim, parecem menores quando a cena é renderizada



Rasterização

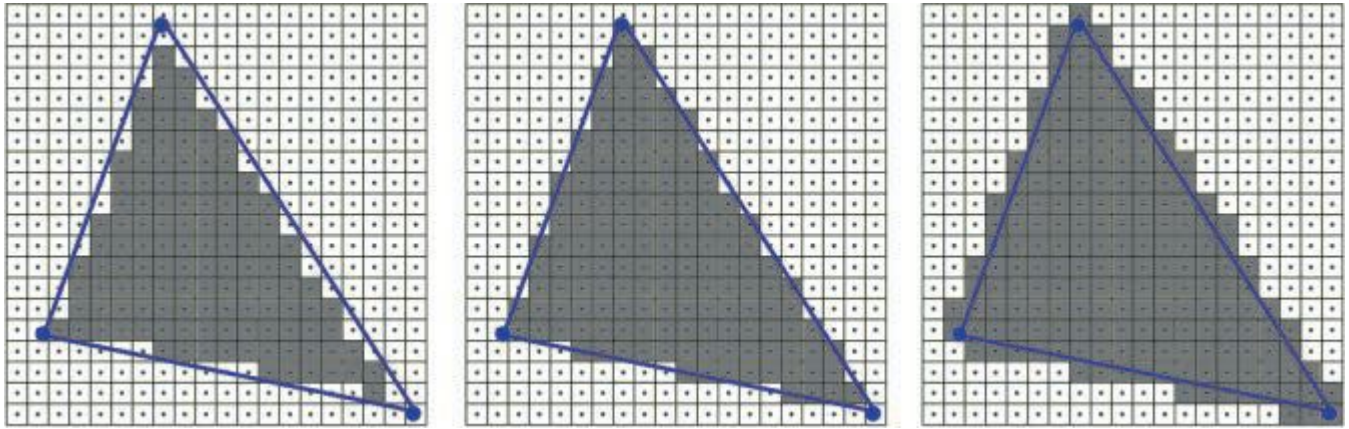
Determina quais pixels **correspondem** às f**ormas geométricas** a serem **renderizadas**

Rasterização

- Os pontos devem primeiro ser agrupados no tipo desejado de primitiva geométrica: pontos, linhas ou triângulos, que consistem em conjuntos de 1, 2 ou 3 pontos

- O próximo passo é determinar quais pixels correspondem ao interior de cada primitiva geométrica
  a. A área inteira do pixel está contida dentro da forma.
  b. O ponto central do pixel está contido dentro da forma
  c. Qualquer parte do pixel está contida dentro da forma

Rasterização

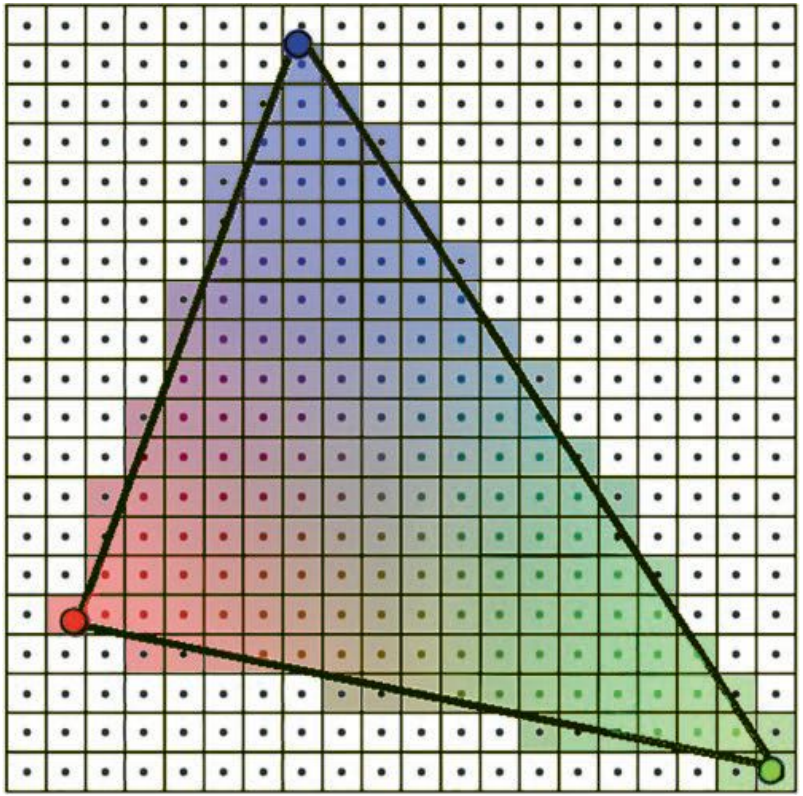


Rasterização

- Dados adicionais podem ser atribuídos a cada vértice, como uma cor, o valor atribuído a este campo em cada ponto interior é interpolado a partir dos valores nos vértices

- Este processo de interpolação garante que as transições de cores entre os vértices sejam suaves, proporcionando um gradiente natural que melhora a aparência visual da cena renderizada

Rasterização



- Determina a cor de cada pixel na imagem renderizada, envolvendo um programa chamado **fragment shader**

Processamento de Pixel

- Os valores de profundidade armazenados em cada fragmento são usados nesta etapa para resolver questões de visibilidade em uma cena tridimensional, determinando quais partes dos objetos estão **bloqueadas da vista por outros objetos**

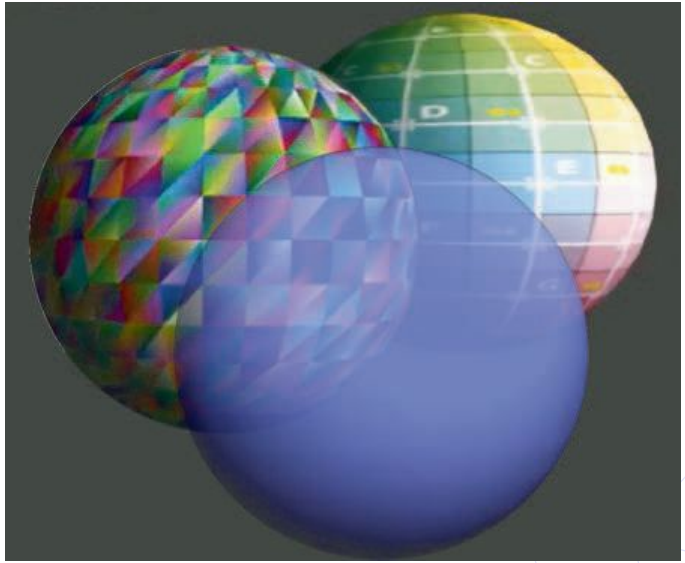
Processamento de Pixel

- Se o fragmento estiver **mais próximo do observador** (profundidade menor), ele **substitui** o pixel armazenado no **buffer de cor**.

- Caso contrário, o fragmento é **descartado**, mantendo o pixel que já estava registrado.

Processamento de Pixel

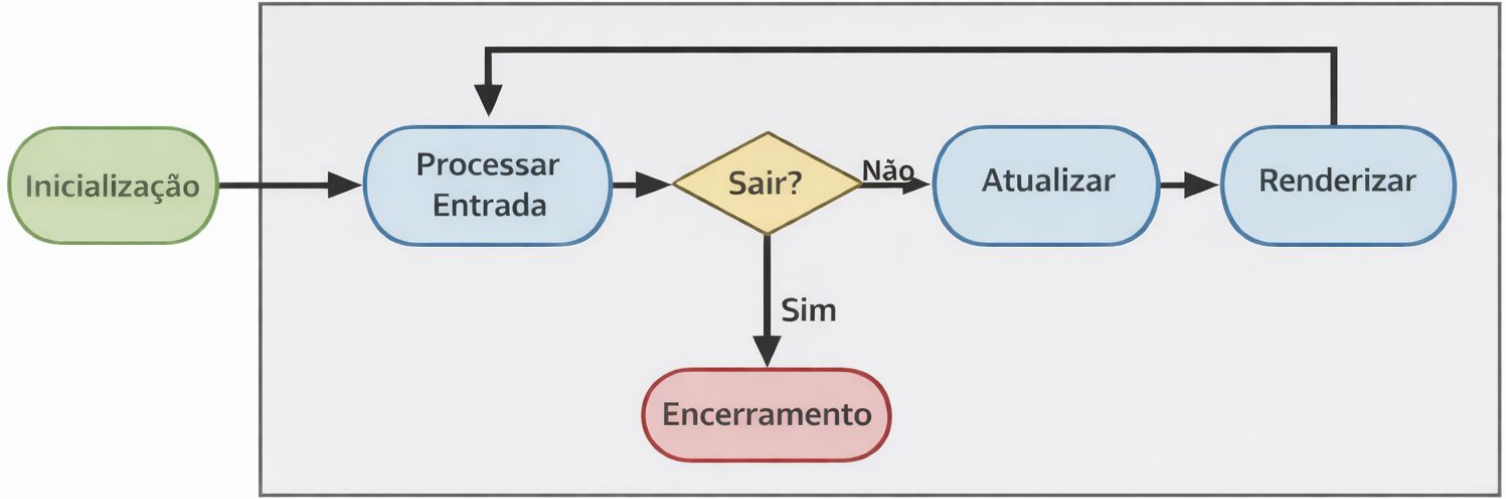
- Cálculo dos valores de transparência, todos os objetos opacos devem ser renderizados primeiro (em qualquer ordem), seguidos por objetos transparentes ordenados do mais distante para o mais próximo em relação à câmera virtual



Setup

- PyCharm Community Edition (python 3.12)
- Bibliotecas glfw e OpenGL

Setup


Inicialização
Processar Entrada
Sair?
Não
Atualizar
Renderizar
Sim
Encerramento

GLFW

- Criação de janelas de contexto de renderização
- Tratamento de eventos de entrada
- Pode ser utilizada em várias APIs
- **pip install glfw**
- https://www.glfw.org/

GLFW - Comandos

init() -> Inicializa a biblioteca GLFW. Deve ser chamado antes de qualquer outra função GLFW

create_window(640, 480, "Minha janela", None, None) -> Cria a janela

make_context_current(window) -> Define a janela window como o contexto **OpenGL ativo**

swap_buffers(window) -> Troca os buffers da janela window, atualizando a tela

GLFW - Comandos

poll_events() -> Processa eventos da janela, como entrada de teclado, mouse e fechamento

window_should_close = Loop -> Loop que processa os frames

Vamos para o exemplo...

GLFW

Vamos continuar, o que falta agora?

OpenGL

- API para renderizar as imagens gráficas
- **pip install PyOpenGL PyOpenGL_accelerate**
- https://pypi.org/project/PyOpenGL/

OpenGL

glClear = Limpar o buffer

- OpenGL gerencia várias "buffers" que armazenam diferentes tipos de dados para renderização.
- GL_COLOR_BUFFER_BIT = Cor
- GL_DEPTH_BUFFER_BIT = Profundidade (z-buffer)
- GL_ACCUM_BUFFER_BIT = Anti-aliasing, motion blur
- GL_STENCIL_BUFFER_BIT = Restringir a certas porções da tela

OpenGL

glClearColor é usada para definir a cor usada quando o buffer de cor é limpo

	R	G	B
red	1	0	0
orange	1	0.5	0
yellow	1	1	0
green	0	1	0
blue	0	0	1
violet	0.5	0	1

	R	G	B
black	0	0	0
white	1	1	1
gray	0.5	0.5	0.5
brown	0.5	0.2	0
pink	1	0.5	0.5
cyan	0	1	1

OpenGL

Vamos continuar o exemplo

Exercício

1) Criar uma janela no tamanho de 800x600
2) Após pressionar a tecla ESQ finalizar o programa
3) Troque randomicamente a cor de fundo da janela, para ser mais perceptível a mudança pode ser utilizado um timer

Referências

- AZEVEDO, Eduardo; CONCI, Aura. Computação gráfica: teoria e prática. Rio de Janeiro: Elsevier, 2003. 353 p.;

- AZEVEDO, Eduardo; CONCI, Aura; LETA, Fabiana R. Computação gráfica: teoria e prática. Rio de Janeiro: Elsevier, 2008. v.

- FOLEY, James D. et al. Computer graphics: principles and practice. 2nd ed. Reading: Addison-Wesley, c1990. 1174 p;

- GONZALEZ, Rafael C.; WOODS, Richard E. Processamento de imagens digitais. São Paulo: E. Blücher, 2000. 509 p.